

TP Cassandra

Cours + Pratique

De zéro à un projet fil rouge CityFlow, avec cassandra-web comme interface

Module	NoSQL — B3 INFO Tronc Commun
Séance	Séance 4 (Jour 2 après-midi)
Durée estimée	4 à 5 heures (cours + TP)
Outils requis	Docker Desktop, navigateur web, éditeur de texte
Public	Étudiants B3 INFO (INFRES + CYBSEC)
Pré-requis	TP MongoDB + Redis terminés, SQL solide
Formateur	KAKE Abdoulaye
Année	2025 / 2026

Comment utiliser ce document ? Ce TP combine du cours et de la pratique en alternance. Les passages « COURS » présentent les concepts. Les passages « VOTRE TOUR » sont des exercices à faire individuellement ou en binôme. Les passages « CityFlow » alimentent le projet fil rouge. Cassandra a sa propre philosophie : prenez votre temps avec les concepts.

Cassandra : origines et philosophie

Une naissance à Facebook

Cassandra a été créée en 2008 chez **Facebook** par **Avinash Lakshman** (l'un des auteurs du paper Amazon Dynamo) et **Prashant Malik** pour répondre à un besoin spécifique : indexer les recherches dans les boîtes de réception de millions d'utilisateurs. Aucune base existante n'était capable de tenir la charge avec une latence acceptable.

Facebook a open-sourcé Cassandra en 2008, et Apache l'a adopté en 2009 comme projet top-level en 2010. Aujourd'hui Cassandra est utilisée par **Netflix, Apple, Instagram, Uber, eBay** et des milliers d'autres pour des charges massives (Apple opère plus de 100 000 nœuds Cassandra).

Le mélange BigTable + Dynamo

Cassandra est une **fusion conceptuelle de deux papers fondateurs** publiés à un an d'intervalle :

Du paper BigTable (Google 2006)	Du paper Dynamo (Amazon 2007)
Modèle de données en familles de colonnes	Architecture peer-to-peer (pas de master)
SSTable et stockage orienté colonne	Consistent hashing pour la distribution
Compactage et compression	Tunable consistency (lecture/écriture)
Notion de partition key	Réplication multi-datacenter

Positionnement technique

Modèle	Wide-column store (familles de colonnes)
Position CAP	AP — disponibilité prioritaire sur cohérence forte (avec tunable consistency)
Architecture	Décentralisée peer-to-peer : tous les nœuds sont égaux, pas de SPOF
Scalabilité	Horizontale linéaire : doubler le cluster double la capacité (jusqu'à des milliers de nœuds)
Performance	Optimisé pour les écritures massives (millions d'écritures/sec sur de gros clusters)
Langage	CQL (Cassandra Query Language), syntaxe proche de SQL mais sémantique différente
Cas typiques	Time-series IoT, logs, métriques, historiques massifs, données distribuées géographiquement

Version actuelle

Cassandra 4.1 (utilisée dans ce cours), 5.0 GA en septembre 2024

Attention : Cassandra n'est PAS « MongoDB en mieux » ni « SQL distribué ». C'est une base avec ses propres règles et son propre paradigme. Ne pas comprendre cette spécificité est la source de la plupart des échecs de projets Cassandra.

Partie 1 — Mise en route avec Docker et cassandra-web

1.1. Objectif de cette partie

À la fin de cette partie, vous aurez **Cassandra qui tourne dans Docker** et **cassandra-web** accessible depuis votre navigateur. Vous ajouterez ces deux services à votre stack CityFlow existante.

1.2. Pourquoi Cassandra ? Pourquoi cassandra-web ?

Cassandra est une base **orientée colonnes** créée pour gérer des volumes massifs de données distribuées. Là où MongoDB et Redis tournent typiquement sur 1 ou 2 nœuds, Cassandra est conçue pour des clusters de 10, 100, voire 1000 machines.

cassandra-web (ipushc/cassandra-web) est une interface web open-source légère pour Cassandra. Elle vous permet de naviguer dans les keyspaces, parcourir les tables, exécuter des requêtes CQL et visualiser les résultats dans le navigateur — sans installer d'application desktop. C'est l'équivalent fonctionnel de mongo-express pour Cassandra.

Note historique : un outil très populaire historiquement s'appelait **DataStax Studio**, mais son image officielle Docker a été retirée du Docker Hub courant 2024-2025. Nous utilisons donc ipushc/cassandra-web, l'alternative communautaire la plus stable. Fonctionnellement, le résultat pédagogique est identique.

Particularité importante : Cassandra met environ **60 secondes** à démarrer (le temps qu'elle bootstrap son cluster, même mono-nœud). cassandra-web démarre rapidement mais attend que Cassandra soit prête via un healthcheck. Patience au premier docker compose up !

1.3. Étape 1 : continuer avec votre dossier cityflow-nosql

```
cd ./cityflow-nosql
docker compose ps # voir l'état actuel
```

Vous devriez voir vos 4 conteneurs précédents (Mongo, mongo-express, Redis, redis-commander) à l'état Up. Si la stack est arrêtée, pas de souci : on relancera tout après modification.

1.4. Étape 2 : ajouter Cassandra et cassandra-web au docker-compose.yml

Ouvrez votre docker-compose.yml. Après redis-commander, **avant la section `volumes`**, ajoutez les deux nouveaux services :

```
cassandra:
  image: cassandra:4.1
  container_name: cityflow-cassandra
  restart: unless-stopped
  ports:
    - "9042:9042"
  volumes:
```

```

- cassandra_data:/var/lib/cassandra
environment:
  CASSANDRA_CLUSTER_NAME: "CityFlow"
  CASSANDRA_DC: "dc1"
  CASSANDRA_ENDPOINT_SNITCH: "GossipingPropertyFileSnitch"
  MAX_HEAP_SIZE: "512M"
  HEAP_NEWSIZE: "128M"
healthcheck:
  test: ["CMD-SHELL", "cqlsh -e 'describe cluster'"]
  interval: 30s
  timeout: 10s
  retries: 10
  start_period: 60s
networks:
  - cityflow-net

cassandra-web:
  image: ipushc/cassandra-web
  container_name: cityflow-cassandra-web
  restart: unless-stopped
  ports:
    - "8083:8083"
  environment:
    CASSANDRA_HOST_IP: cassandra
    CASSANDRA_PORT: 9042
    HOST_PORT: ":8083"
  depends_on:
    cassandra:
      condition: service_healthy
  networks:
    - cityflow-net

```

Modifiez la section volumes pour ajouter cassandra_data :

```

volumes:
  mongo_data:
  redis_data:
  cassandra_data:

```

Comprendre la configuration Cassandra

- ``image: cassandra:4.1`` : version 4.1, stable et bien supportée
- ``9042:9042`` : port CQL standard utilisé par les drivers et cassandra-web
- ``CASSANDRA_CLUSTER_NAME`` : nom logique du cluster, visible dans les outils
- ``MAX_HEAP_SIZE: 512M`` : limite la JVM Cassandra à 512 Mo (pour économiser la RAM en local)
- ``healthcheck`` : vérifie que Cassandra est vraiment prête
- ``start_period: 60s`` : on laisse Cassandra 60s avant de la considérer en échec

Comprendre la configuration cassandra-web

- ``image: ipushc/cassandra-web`` : image communautaire, légère (~50 Mo)
- ``8083:8083`` : port HTTP de l'interface
- ``CASSANDRA_HOST_IP: cassandra`` : connexion au conteneur Cassandra par son nom (pas localhost !)
- ``HOST_PORT: ":8083"`` : port d'écoute (notation Go avec deux-points devant)
- ``condition: service_healthy`` : attend que Cassandra soit prête avant de démarrer

⚠ **RAM nécessaire** : Cassandra + cassandra-web consomment environ **2 Go de RAM** en plus de ce qui tourne déjà. Si votre machine a 8 Go, ça devrait passer. Si vous avez 4 Go, arrêtez d'abord les autres services lourds.

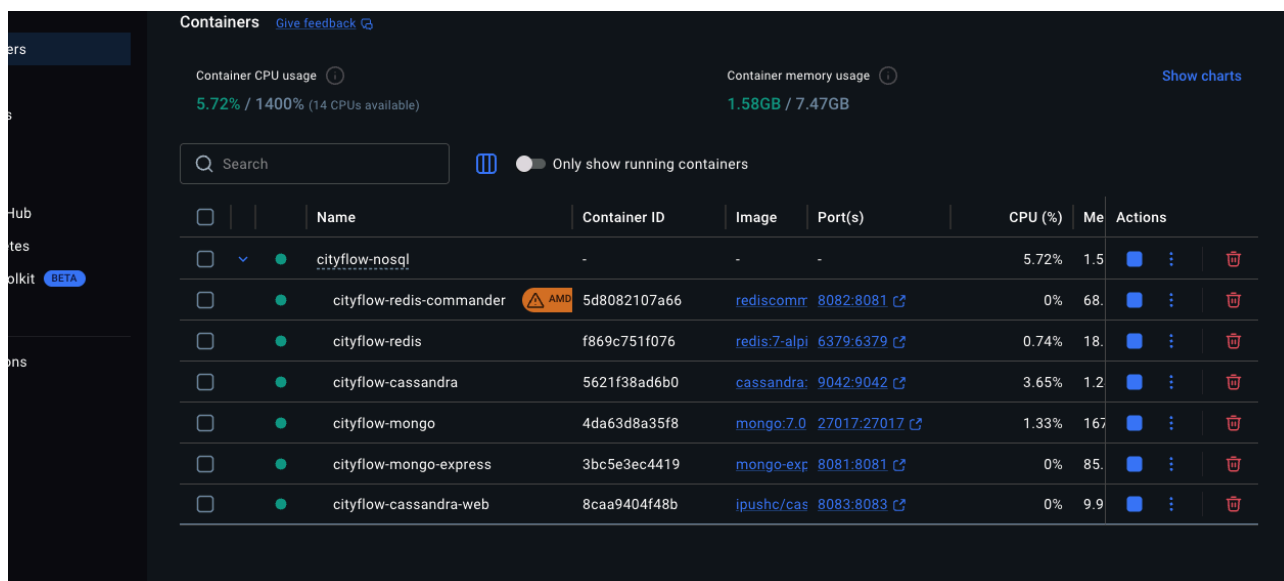
1.5. Étape 3 : lancer la stack étendue

```
# Relancer la stack
docker compose up -d

# Suivre les logs Cassandra le temps du démarrage (env. 60s)
docker compose logs -f cassandra
# Attendez le message "Starting listening for CQL clients on /0.0.0.0:9042"
# Puis CTRL+C pour quitter les logs

# Vérifier l'état
docker compose ps
```

Tous vos conteneurs doivent être Up. Cassandra peut être en starting pendant 1-2 minutes, c'est normal. cassandra-web n'apparaîtra comme Up qu'après le healthcheck Cassandra passé.



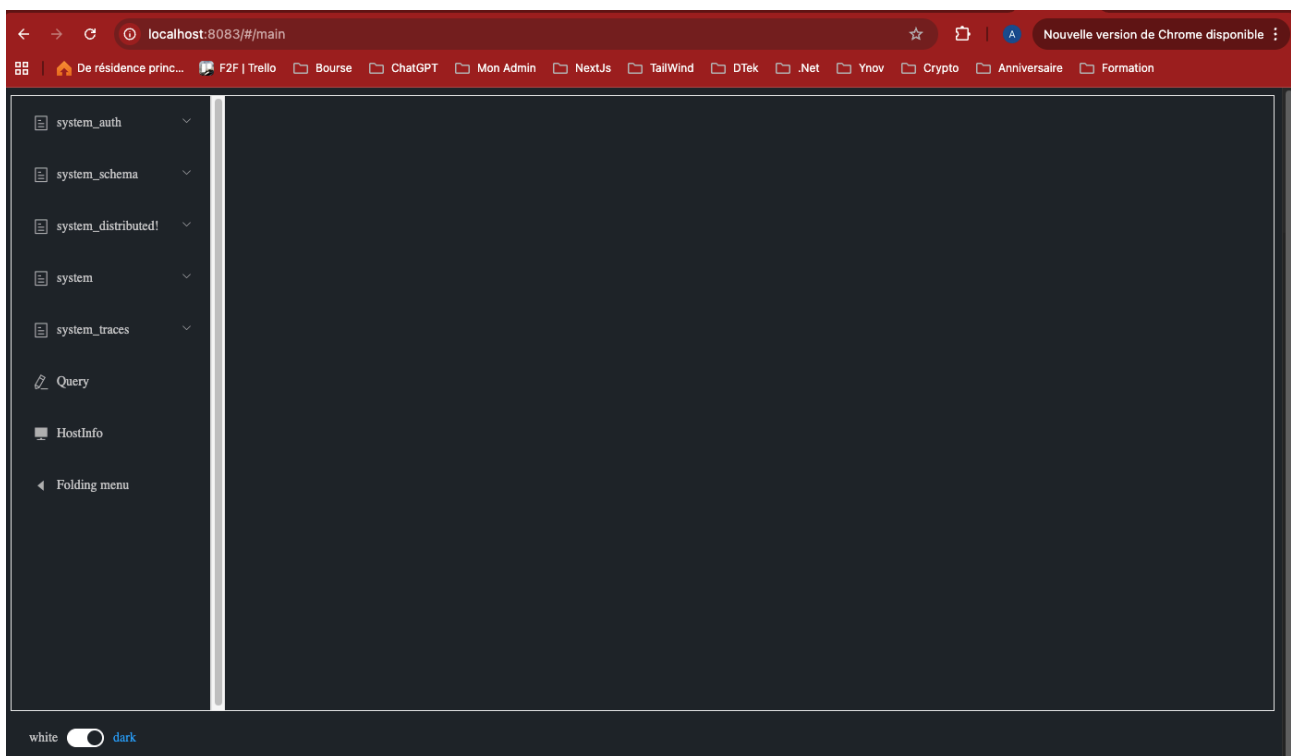
The screenshot shows the Docker Desktop interface with the 'Containers' tab selected. It displays a list of running containers for the 'cityflow' stack. The containers are: cityflow-nosql, cityflow-redis-commander, cityflow-redis, cityflow-cassandra, cityflow-mongo, cityflow-mongo-express, and cityflow-cassandra-web. Each container shows its name, ID, image, ports, CPU usage, memory usage, and actions (stop, restart, delete).

Name	Container ID	Image	Port(s)	CPU (%)	Me	Actions
cityflow-nosql	-	-	-	5.72%	1.5	[Stop] [Restart] [Delete]
cityflow-redis-commander	5d8082107a66	rediscommr	8082:8081	0%	68.	[Stop] [Restart] [Delete]
cityflow-redis	f869c751f076	redis:7-alpi	6379:6379	0.74%	18.	[Stop] [Restart] [Delete]
cityflow-cassandra	5621f38ad6b0	cassandra	9042:9042	3.65%	1.2	[Stop] [Restart] [Delete]
cityflow-mongo	4da63d8a35f8	mongo:7.0	27017:27017	1.33%	167	[Stop] [Restart] [Delete]
cityflow-mongo-express	3bc5e3ec4419	mongo-exp	8081:8081	0%	85.	[Stop] [Restart] [Delete]
cityflow-cassandra-web	8caa9404f48b	ipushc/cas	8083:8083	0%	9.9	[Stop] [Restart] [Delete]

1.6. Étape 4 : ouvrir cassandra-web

Dans votre navigateur, ouvrez : <http://localhost:8083>

L'interface cassandra-web s'ouvre directement, sans authentification (Cassandra n'a pas d'auth activée pour ce TP). Vous voyez immédiatement la liste des **keyspaces** existants.

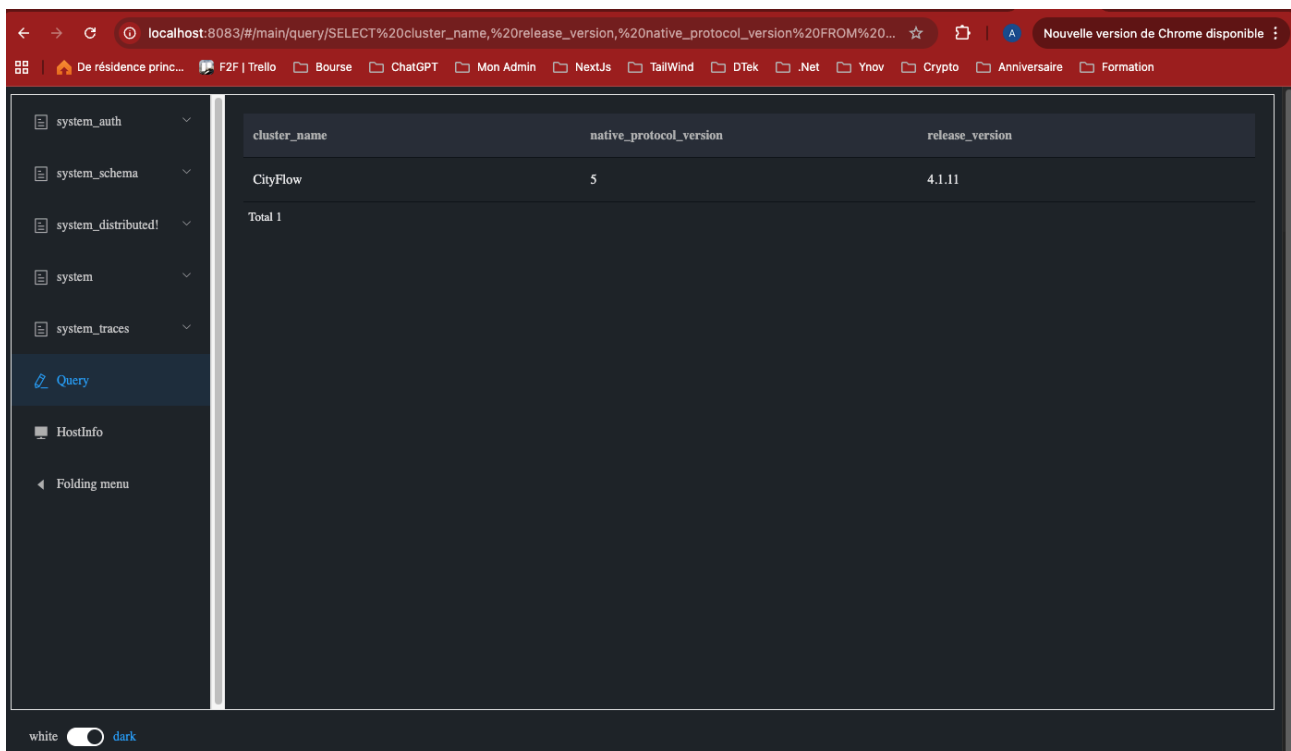


1.7. Étape 5 : tester une première requête

cassandra-web propose une **console CQL** intégrée. Cherchez le bouton **Query** ou **Execute** (selon la version, situé en haut de l'interface). Tapez :

```
SELECT cluster_name, release_version, native_protocol_version FROM system.local;
```

Vous devez voir un tableau avec les infos du cluster CityFlow. **Si oui, votre Cassandra est opérationnelle.**



VOTRE TOUR — Validation de la mise en route

1. docker compose ps montre **6 conteneurs Up** (Mongo, mongo-express, Redis, redis-commander, Cassandra, cassandra-web).
2. <http://localhost:8081> (mongo-express) et <http://localhost:8082> (redis-commander) marchent toujours.
3. <http://localhost:8083> (cassandra-web) ouvre l'interface.
4. Vous voyez la liste des keyspaces système (system, system_schema, etc.).
5. Votre première requête `SELECT FROM system.local` renvoie les infos du cluster.

Partie 2 — Découverte de l'interface cassandra-web

2.1. Anatomie de l'interface

Vue d'ensemble

cassandra-web est volontairement minimaliste — elle se concentre sur l'essentiel pour explorer une base Cassandra :

- **Liste des keyspaces** à gauche (ou en page d'accueil selon version)
- **Liste des tables** d'un keyspace sélectionné
- **Visualisation des données** d'une table en mode tableau
- **Console CQL** pour exécuter n'importe quelle requête
- **Schéma de la table** (colonnes, types, clés primaires)

Les keyspaces système

Au démarrage, Cassandra a déjà plusieurs keyspaces système — vous les voyez listés :

Keyspace	Rôle
system	Métadonnées internes de Cassandra (ne pas toucher)
system_schema	Schémas des tables (ne pas toucher)
system_auth	Authentification (vide ici)
system_distributed	Coordination interne
system_traces	Traces de requêtes (pour le debug)

Nous allons créer notre propre keyspace `tp_cassandra` dans la partie 6. Mais d'abord, il faut comprendre le modèle Cassandra.

2.2. Premières commandes utiles

Dans la console CQL de cassandra-web, exécutez ces commandes une par une :

```
-- Voir tous les keyspaces
DESCRIBE KEYSPACES;

-- Détails du keyspace système
DESCRIBE KEYSPACE system;

-- Détails d'une table système
DESCRIBE TABLE system.local;

-- Statistiques du cluster
SELECT cluster_name, data_center, rack FROM system.local;
```

VOTRE TOUR — Exploration libre

Sans rien créer ni modifier, explorez cassandra-web pendant 10 minutes :

1. Naviguez dans tous les keyspaces système.
2. Cliquez sur quelques tables (par exemple `system.local`, `system_schema.tables`) pour voir leur structure.
3. Observez les types de colonnes utilisés.
4. Exécutez les commandes du paragraphe 2.2 dans la console.
5. Notez sur papier : qu'est-ce qui vous semble différent par rapport à mongo-express ou redis-commander ?

Partie 3 — Cours : architecture et modèle de données Cassandra

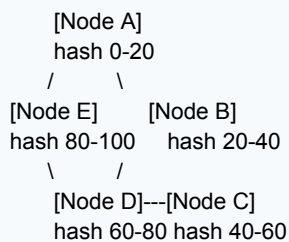
3.1. Architecture peer-to-peer : pas de master

Contrairement à PostgreSQL ou MongoDB (qui ont un nœud primaire), **Cassandra n'a pas de master**. Tous les nœuds sont identiques et peuvent recevoir n'importe quelle requête. C'est cette absence de point unique de défaillance qui fait sa robustesse.

Le cluster en anneau

Les nœuds Cassandra sont organisés logiquement en **anneau (ring)**. Chaque nœud est responsable d'une plage de l'espace de hachage. Quand une donnée arrive, son hash détermine sur quel nœud elle ira.

Cluster Cassandra à 5 nœuds (vue logique) :



- Quand une donnée arrive, son hash détermine sur quel nœud elle va.
- Chaque nœud parle à tous les autres via le protocole Gossip.
- Pas de master, pas de SPOF (Single Point of Failure).

Le protocole Gossip

Comment des nœuds peer-to-peer se coordonnent-ils sans master ? Via le **protocole Gossip** (« commérage » en anglais). Chaque seconde, chaque nœud choisit aléatoirement 1-3 autres nœuds et leur partage son état. Les rumeurs se propagent exponentiellement et l'ensemble du cluster converge vers une vue cohérente en quelques secondes.

Conséquence pratique : Cassandra encaisse très bien la perte de nœuds. Si 1 nœud sur 10 tombe, le cluster continue à servir les requêtes (réplication oblige), et les autres nœuds sont au courant en quelques secondes via le Gossip.

3.3. La réplication et la tunable consistency

Chaque donnée est **répliquée** sur plusieurs nœuds (typiquement 3) pour garantir la disponibilité. Le facteur de réplication (RF) est défini par keyspace.

```
-- Création d'un keyspace avec réplication x3
CREATE KEYSPACE cityflow
WITH replication = {
  'class': 'SimpleStrategy',
  'replication_factor': 3
};

-- Multi-datacenter
CREATE KEYSPACE cityflow
WITH replication = {
  'class': 'NetworkTopologyStrategy',
  'datacenter_lyon': 3,
  'datacenter_paris': 3
};
```

Tunable consistency : vous choisissez le compromis

Cassandra propose un compromis **ajustable** entre cohérence et disponibilité, configurable **par requête**. Vous décidez combien de réplicas doivent répondre pour qu'une opération soit considérée comme réussie :

Niveau	Lecture	Écriture	Cas d'usage
ONE	1 réplica suffit	1 réplica acquitte	Max de perf, tolérant aux incohérences
QUORUM	Majorité des réplicas	Majorité acquitte	Compromis perf/cohérence (recommandé)
ALL	Tous les réplicas	Tous acquittent	Cohérence forte, mais sensible aux pannes
LOCAL_QUORUM	Quorum dans le DC local	Idem	Multi-DC : éviter latence inter-DC

Théorème de la cohérence forte : si $R + W > RF$, vous avez cohérence forte (où R = consistency level lecture, W = écriture, RF = facteur de réplication). Avec $RF=3$: QUORUM en lecture ($R=2$) + QUORUM en écriture ($W=2$) = $4 > 3 \rightarrow$ cohérence garantie.

3.4. La hiérarchie de données Cassandra

```
Cluster Cassandra
├── Keyspace (équivalent d'une "database")
│   ├── Table (équivalent SQL avec règles différentes)
│   │   ├── Partition (un ensemble de lignes regroupées sur 1 nœud)
│   │   │   ├── Row (ligne)
│   │   │   │   ├── Columns (colonnes)
```

Le keyspace

Le keyspace est le conteneur de premier niveau, équivalent d'une « database » en SQL. C'est au niveau du keyspace que se définit la **stratégie de réplication**.

La table

Une table Cassandra ressemble à une table SQL avec ses lignes et colonnes, **mais elle a des règles très différentes** :

- Toutes les colonnes ne doivent pas être présentes dans toutes les lignes (schéma flexible)
- La PRIMARY KEY a une structure spéciale (vu dans la partie 4)
- Les requêtes sont fortement contraintes par la définition de la table
- Pas de jointures, pas de transactions multi-lignes par défaut

Exemple de table CityFlow

```
CREATE TABLE station_passages (
  station_id text,
  day date,
  timestamp timestamp,
  velos_dispo int,
  scooters_dispo int,
  PRIMARY KEY ((station_id, day), timestamp)
) WITH CLUSTERING ORDER BY (timestamp DESC);

-- Décodage :
-- - (station_id, day) entre parenthèses = partition key composée
-- - timestamp = clustering key
```

-- - Toutes les colonnes sont stockées physiquement par "famille"
-- - WITH CLUSTERING ORDER BY = tri physique sur le disque

3.5. Types de données CQL principaux

Type	Usage
text / varchar	Chaîne UTF-8
int / bigint	Entiers 32 ou 64 bits
float / double	Nombres à virgule
boolean	true / false
uuid / timeuuid	Identifiants uniques (timeuuid intègre un timestamp)
date / time / timestamp	Date, heure, ou les deux
blob	Données binaires brutes
list<T> / set<T> / map<K,V>	Collections (utilisez avec modération)
counter	Compteur incrémental atomique (table dédiée)

Partie 4 — Cours : PRIMARY KEY et query-first design

C'est LE chapitre central de Cassandra. Mal le comprendre = garantir des modélisations catastrophiques et des problèmes de performance. Prenez votre temps, relisez si besoin.

4.1. Anatomie d'une PRIMARY KEY

Une PRIMARY KEY Cassandra a une structure spéciale en **deux parties** :

```
CREATE TABLE station_passages (  
  station_id text,      -- partie 1 de la PARTITION KEY  
  day date,            -- partie 2 de la PARTITION KEY (composée)  
  timestamp timestamp, -- CLUSTERING KEY  
  velos_dispo int,  
  scooters_dispo int,  
  PRIMARY KEY (  
    (station_id, day), -- entre parenthèses = composite partition key  
    timestamp         -- après = clustering key  
  )  
);
```

4.2. La partition key : sur QUEL nœud ?

La **partition key** détermine sur quel nœud du cluster une ligne sera physiquement stockée. Cassandra applique une fonction de hachage (Murmur3 par défaut) à la partition key pour calculer un token, qui est ensuite localisé sur l'anneau.

Conséquence majeure : toutes les lignes ayant la même partition key sont stockées physiquement sur le même nœud, côte à côte. C'est ce qui rend les requêtes ciblées extrêmement rapides : Cassandra sait exactement où aller chercher les données.

-- Exemple : table de passages
-- partition key = (station_id, day)

```
INSERT INTO station_passages VALUES ('S001', '2026-05-15', '2026-05-15 14:32:00', 12, 5);
INSERT INTO station_passages VALUES ('S001', '2026-05-15', '2026-05-15 14:35:00', 11, 5);
INSERT INTO station_passages VALUES ('S001', '2026-05-16', '2026-05-15 08:10:00', 10, 4);
INSERT INTO station_passages VALUES ('S002', '2026-05-15', '2026-05-15 14:32:00', 8, 3);
```

- → On a 3 partitions distinctes :
- 1. (S001, 2026-05-15) : 2 lignes sur le nœud N_a
- 2. (S001, 2026-05-16) : 1 ligne sur le nœud N_b
- 3. (S002, 2026-05-15) : 1 ligne sur le nœud N_c
- Une requête sur (S001, 2026-05-15) ira tout droit au nœud N_a → ultra-rapide
- Une requête sans partition key DOIT scanner tous les nœuds → catastrophe

4.3. La clustering key : dans QUEL ORDRE ?

La **clustering key** détermine l'ordre des lignes **au sein d'une partition**. C'est elle qui permet de récupérer rapidement des intervalles ou de trier le résultat sans opération coûteuse.

- Avec CLUSTERING ORDER BY (timestamp DESC),
- les lignes d'une partition sont stockées triées par timestamp décroissant.
- Requête ultra-rapide : 10 derniers passages d'une station un jour donné
- SELECT * FROM station_passages
- WHERE station_id = 'S001' AND day = '2026-05-15'
- LIMIT 10;
- → Cassandra lit séquentiellement les 10 premières lignes de la partition.
- → O(1) en accès, lecture séquentielle ultra-rapide.
- Intervalle dans le temps
- SELECT * FROM station_passages
- WHERE station_id = 'S001' AND day = '2026-05-15'
- AND timestamp >= '2026-05-15 14:00:00'
- AND timestamp < '2026-05-15 15:00:00';

4.4. Comment choisir une partition key ?

Critère	Implication
Cardinalité élevée	Beaucoup de partitions différentes = bonne distribution sur le cluster
Taille max bornée	Une partition ne doit pas dépasser ~100 MB
Toujours fournie dans les requêtes	Toutes les requêtes principales doivent utiliser la partition key
Distribution uniforme	Évitez les partition keys qui créent des hot spots

Le pattern « bucketing par temps » : ajouter day (ou year_month) à la partition key évite que les partitions grossissent indéfiniment. Une partition par station par jour reste bornée à quelques milliers de lignes max.

4.5. Query-first design : le saut mental

En SQL, on modélise les données (normalisation), puis on écrit n'importe quelle requête.
En Cassandra, on liste d'abord les requêtes, puis on conçoit les tables qui les serviront.
C'est le renversement le plus important à intégrer. Les développeurs SQL résistent souvent à ce changement de paradigme — résistez à la résistance !

Le renversement en tableau

SQL relationnel	Cassandra
1. Normaliser les données (1NF, 2NF, 3NF)	1. Lister les requêtes que l'application va faire
2. Écrire n'importe quelle requête avec JOIN	2. Créer une table par type de requête
3. Compter sur l'optimiseur pour la performance	3. Accepter la duplication de données
4. Cohérence garantie par les contraintes	4. Cohérence gérée par l'application (writes synchrones sur plusieurs tables)

La démarche en 4 étapes

- **Lister les requêtes** que l'application va exécuter (« quelles questions posera-t-on aux données ? »)
- **Identifier les patterns d'accès** : quelles colonnes en filtre, quel tri, quelle granularité ?
- **Concevoir une table par requête** avec la PRIMARY KEY adaptée
- **Accepter la duplication** : une même donnée peut être stockée dans plusieurs tables

Étude de cas CityFlow

Étape 1 — Identifier les requêtes principales sur les passages de stations :

- **Q1** : Récupérer les passages d'une station donnée pour une date donnée
- **Q2** : Récupérer tous les passages d'un utilisateur (audit) sur les 30 derniers jours
- **Q3** : Statistiques journalières par station

Étape 2 — Une table par requête :

```
-- Table pour Q1 : station_passages
CREATE TABLE station_passages (
  station_id text, day date, timestamp timestamp,
  user_id text, event_type text,
  PRIMARY KEY ((station_id, day), timestamp)
) WITH CLUSTERING ORDER BY (timestamp DESC);

-- Table pour Q2 : user_passages
CREATE TABLE user_passages (
  user_id text, day date, timestamp timestamp,
  station_id text, event_type text,
  PRIMARY KEY ((user_id), day, timestamp)
) WITH CLUSTERING ORDER BY (day DESC, timestamp DESC);

-- Table pour Q3 : daily_station_stats (pré-agrégée)
CREATE TABLE daily_station_stats (
  station_id text, day date,
  total_passages counter,
  PRIMARY KEY (station_id, day)
);
```

Étape 3 — Accepter la duplication :

Un même passage utilisateur sera écrit dans **3 tables différentes**. C'est volontaire ! Le stockage est bon marché, la performance de lecture est ce qui compte. L'application est responsable d'écrire dans les 3 tables en cohérence (idéalement via un BATCH).

Partie 5 — Exercices de modélisation papier

5.1. La méthode

Pour chaque exercice, sur une feuille blanche :

- **Listez les requêtes** que l'application devra faire
- **Identifiez les patterns d'accès** : filtre, tri, granularité
- **Dessinez une table par requête** avec sa PRIMARY KEY
- **Pour chaque PRIMARY KEY**, justifiez : pourquoi cette partition key, pourquoi cette clustering key ?

VOTRE TOUR — Exercice 1: IoT capteurs météo

Vous gérez 5 000 capteurs météo répartis en France. Chaque capteur envoie une mesure par minute (température, humidité, pression).

Requêtes attendues :

- Q1 : Historique d'un capteur sur un mois
- Q2 : Mesures les plus récentes (les 10 dernières) d'un capteur
- Q3 : Toutes les mesures d'une journée pour une région donnée

Travail demandé :

1. Calculez le **volume de données par an** : $5000 \text{ capteurs} \times 60 \text{ mesures/h} \times 24 \text{h} \times 365 \text{j}$.
2. Proposez une modélisation avec **bucketing temporel** (partition par jour ou par mois ?).
3. Pour chaque table, justifiez le choix de la partition key.
4. Estimez la **taille d'une partition typique** avec votre modélisation.
5. Bonus : que se passerait-il avec PRIMARY KEY (sensor_id, timestamp) ?

VOTRE TOUR — Exercice 2 : Réseau social messages

Vous gérez les messages privés d'un réseau social. Chaque conversation a 2 utilisateurs et des centaines voire milliers de messages.

Requêtes attendues :

- Q1 : Tous les messages d'une conversation entre 2 users, du plus récent au plus ancien
- Q2 : Les conversations actives d'un utilisateur (par ordre de dernier message)
- Q3 : Recherche d'un message dans une conversation par mot-clé (bonus)

Travail demandé :

1. Modélisez les tables pour Q1 et Q2.
2. Réfléchissez : comment représenter une conversation à deux participants ? Par convention (user1_id < user2_id) ou par un UUID dédié ?
3. Pour la pagination des messages anciens, comment faire ?
4. Pour Q3 (recherche par mot-clé), expliquez **pourquoi Cassandra n'est pas adaptée** et quelle techno utiliser à la place.

VOTRE TOUR — Exercice 3 : E-commerce - logs de visites

Vous loggez chaque visite de page produit sur un e-commerce :

- 100 000 produits
- 1 million de visites par jour, réparties inégalement (certains produits font 10 vues/jour, d'autres 50 000)

Requêtes attendues :

- Q1 : Toutes les visites d'un produit donné sur la dernière semaine
- Q2 : Top 10 des produits les plus visités sur une journée
- Q3 : Toutes les visites d'un utilisateur (audit RGPD)

Travail demandé :

1. Pour Q1, attention aux produits stars qui font 50 000 vues/jour : risquez-vous une partition trop grosse ? Comment l'éviter ?
2. Pour Q2 (top 10 produits/jour), expliquez pourquoi Cassandra n'est pas adaptée à cette requête « pure » et proposez **une alternative** (pré-agrégation, Redis sorted set, ou Spark sur Cassandra).
3. Pour Q3, quelle est votre PRIMARY KEY ?

Partie 6 — Création et insertion guidée (CQL)

6.1. Objectif

Vous allez créer votre premier keyspace, vos premières tables et insérer des données. cassandra-web est votre interface — la console CQL exécute n'importe quelle requête.

6.2. Étape 1 : créer un keyspace

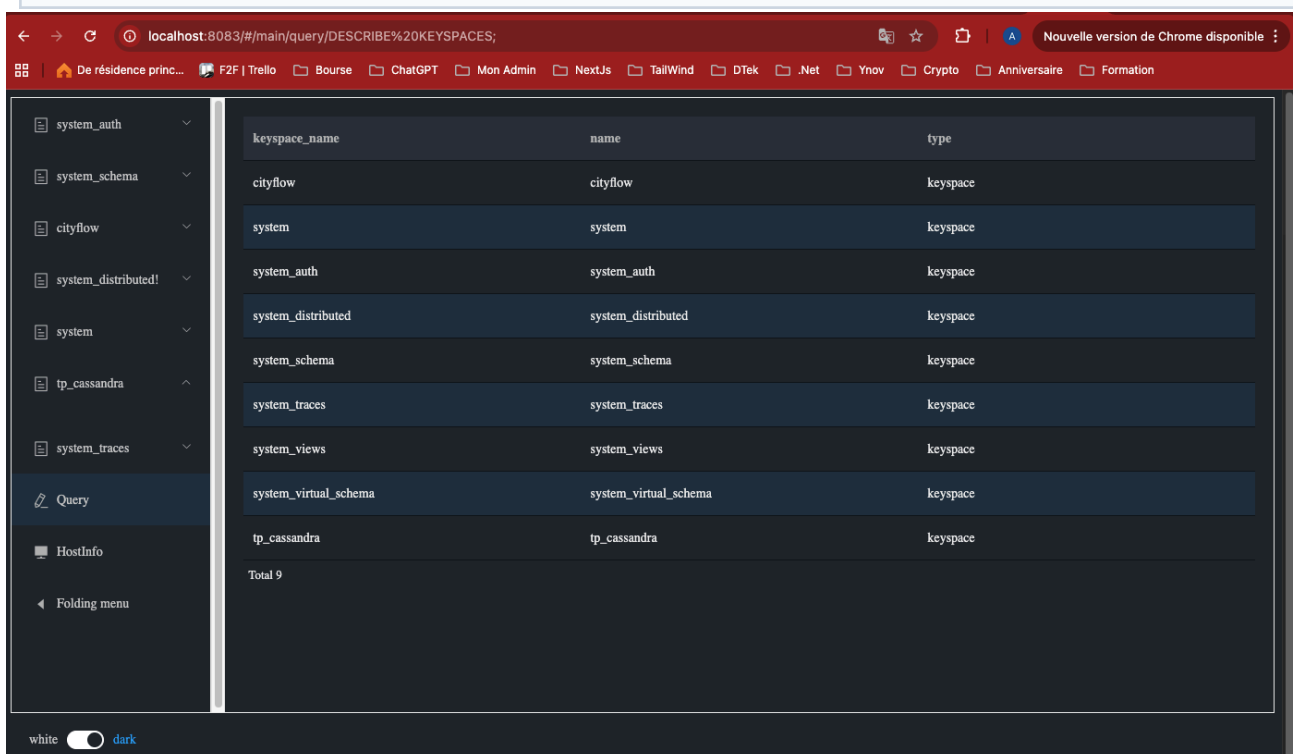
Dans la console CQL de cassandra-web, exécutez :

```
CREATE KEYSPACE IF NOT EXISTS tp_cassandra
WITH replication = {
  'class': 'SimpleStrategy',
  'replication_factor': 1
};
```

Le replication_factor: 1 est une simplification pour le TP local (un seul nœud). En production, on a typiquement 3.

Vérifiez dans une nouvelle cellule :

```
DESCRIBE KEYSPACES;
-- Vous devez voir tp_cassandra dans la liste
```



6.3. Étape 2 : créer votre première table

On va créer une table de profils utilisateurs simple :

```
USE tp_cassandra;

CREATE TABLE tp_cassandra.users (
  user_id uuid PRIMARY KEY,
  email text,
  first_name text,
```

```
last_name text,  
age int,  
city text,  
created_at timestamp,  
tags set<text>  
);
```

Vérifiez la structure :

```
DESCRIBE TABLE tp_cassandra.users;
```

6.4. Étape 3 : insérer des données

Insérez 3 utilisateurs (notez la fonction `uuid()` qui génère un UUID aléatoire) :

```
INSERT INTO tp_cassandra.users (user_id, email, first_name, last_name, age, city, created_at, tags)  
VALUES (uuid(), 'alice@example.com', 'Alice', 'Dupont', 28, 'Lyon', toTimestamp(now()), {'premium', 'fr'});  
  
INSERT INTO tp_cassandra.users (user_id, email, first_name, last_name, age, city, created_at, tags)  
VALUES (uuid(), 'bob@example.com', 'Bob', 'Martin', 32, 'Villeurbanne', toTimestamp(now()), {'newbie'});  
  
INSERT INTO tp_cassandra.users (user_id, email, first_name, last_name, age, city, created_at, tags)  
VALUES (uuid(), 'carol@example.com', 'Carol', 'Lefèvre', 25, 'Lyon', toTimestamp(now()), {'premium', 'eco-friendly'});
```

Vérifiez :

```
SELECT * FROM tp_cassandra.users;
```

Observez : dans `cassandra-web`, le résultat s'affiche en tableau avec des colonnes. Les **types complexes** comme `set<text>` sont affichés directement (par exemple `{'premium', 'fr'}`).

6.5. Étape 4 : observer les contraintes CQL

Tentez les requêtes suivantes et observez les erreurs (c'est volontaire !) :

```
-- Tentative : filtrer par age (sans utiliser la PRIMARY KEY)  
SELECT * FROM tp_cassandra.users WHERE age > 25;  
-- → Erreur : "Cannot execute this query as it might involve data filtering"  
  
-- Solution forcée (à ne JAMAIS faire en production)  
SELECT * FROM tp_cassandra.users WHERE age > 25 ALLOW FILTERING;  
-- → Marche, mais c'est un scan complet de la table
```

Premier choc culturel CQL : `WHERE age > 25` ne marche pas. Pourquoi ? Parce que `age` n'est pas dans la `PRIMARY KEY`, et Cassandra refuse les requêtes qui ne ciblent pas directement une partition. `ALLOW FILTERING` force l'exécution mais c'est un anti-pattern.

Si vous voulez requêter par age, vous devez créer une table dédiée avec une `PRIMARY KEY` adaptée. C'est le `query-first design`.

6.6. Étape 5 : créer une table avec partition + clustering keys

Créons maintenant une table de passages de stations CityFlow :

```
CREATE TABLE cityflow.station_passages (  
  station_id text,  
  day date,  
  passage_time timestamp,  
  user_id text,  
  event_type text,  
  PRIMARY KEY ((station_id, day), passage_time)  
) WITH CLUSTERING ORDER BY (passage_time DESC);
```

Insérez quelques passages :

```
INSERT INTO cityflow.station_passages (station_id, day, passage_time, user_id, event_type)  
VALUES ('S001', '2025-09-15', '2025-09-15 08:30:00', 'user_42', 'rent');  
  
INSERT INTO cityflow.station_passages (station_id, day, passage_time, user_id, event_type)  
VALUES ('S001', '2025-09-15', '2025-09-15 09:45:00', 'user_55', 'rent');  
  
INSERT INTO cityflow.station_passages (station_id, day, passage_time, user_id, event_type)  
VALUES ('S001', '2025-09-15', '2025-09-15 17:20:00', 'user_42', 'return');  
  
INSERT INTO cityflow.station_passages (station_id, day, passage_time, user_id, event_type)  
VALUES ('S002', '2025-09-15', '2025-09-15 10:00:00', 'user_77', 'rent');
```

Maintenant testez la **différence de comportement** :

```
-- Marche : on cible une partition complète  
SELECT * FROM cityflow.station_passages  
WHERE station_id = 'S001' AND day = '2025-09-15';  
  
-- Marche : on cible une partition + intervalle sur clustering key  
SELECT * FROM cityflow.station_passages  
WHERE station_id = 'S001' AND day = '2025-09-15'  
  AND passage_time >= '2025-09-15 09:00:00';  
  
-- Échoue : on ne fournit pas la partition key complète  
SELECT * FROM cityflow.station_passages WHERE station_id = 'S001';  
-- → Erreur  
  
-- Échoue : pas de partition key du tout  
SELECT * FROM cityflow.station_passages WHERE day = '2025-09-15';  
-- → Erreur
```

VOTRE TOUR — Créer votre propre dataset

Étape 1 : créer une table pour vos passages utilisateurs

Créez une table `user_passages` qui répond à la question « historique des passages d'un utilisateur ». Pensez à la partition key et à la clustering key. **Justifiez votre choix sur papier** avant de créer la table.

Étape 2 : créer une table d'agrégation

Créez une table `daily_station_stats` qui stocke un compteur du nombre de passages par (station, jour). Indice : utilisez le type counter.

Étape 3 : peupler vos tables

Insérez au moins 30 passages variés dans `station_passages` : 5 stations différentes, 3-5 jours différents, plusieurs utilisateurs. Pensez à insérer en parallèle dans `user_passages` (duplication volontaire).

Étape 4 : tester les requêtes

Vérifiez que vous pouvez :

- Lister les passages d'une station un jour donné

- Lister les passages d'un utilisateur sur les 30 derniers jours
- Lire le compteur de passages d'une station un jour donné

Partie 7 — TP : requêtes à produire

7.1. Objectif et méthode

Vous allez maintenant **écrire vos propres requêtes CQL**. Toutes les requêtes seront testées dans cassandra-web. **Notez chaque requête** dans un fichier `mes_requetes_cql.md` pour le projet CityFlow.

7.2. Exemples résolus pour démarrer

Exemple 1 : passages les plus récents d'une station

```
SELECT * FROM cityflow.station_passages
WHERE station_id = 'S001' AND day = '2025-09-15'
LIMIT 5;
```

Exemple 2 : intervalle de temps sur une journée

```
SELECT passage_time, user_id, event_type FROM cityflow.station_passages
WHERE station_id = 'S001' AND day = '2025-09-15'
AND passage_time >= '2025-09-15 08:00:00'
AND passage_time < '2025-09-15 12:00:00';
```

Exemple 3 : récupération d'une ligne précise

```
SELECT * FROM cityflow.station_passages
WHERE station_id = 'S001' AND day = '2025-09-15'
AND passage_time = '2025-09-15 08:30:00';
```

VOTRE TOUR — Requêtes CQL à produire

Pour chaque demande, **écrivez la requête CQL** dans une cellule cassandra-web, testez, et notez dans votre fichier `mes_requetes_cql.md`.

Niveau 1 — SELECT basiques (sur vos données existantes)

1. Listez tous les passages de la station S002 le 15/09/2025.
2. Listez les 3 derniers passages de la station S001 le 15/09/2025.
3. Listez les passages entre 9h et 18h pour une station + un jour donnés.
4. Comptez le nombre de passages d'une station un jour donné (COUNT(*)).

Niveau 2 — Requêtes sur user_passages

5. Listez tous les passages d'un utilisateur donné (toutes dates confondues).
6. Listez les passages d'un utilisateur sur les 7 derniers jours.
7. Quels sont les 5 derniers passages d'un utilisateur ?

Niveau 3 — INSERT, UPDATE, DELETE

8. Insérez un nouveau passage à l'instant now() pour une station/utilisateur de votre choix.
9. Mettez à jour un passage existant : changez son event_type de rent à return (indice : utilisez UPDATE, mais en CQL c'est en fait un UPSERT).
10. Supprimez un passage précis (DELETE avec PRIMARY KEY complète).

Niveau 4 — Compteurs

11. Incrémentez le compteur total_passages de daily_station_stats pour S001/aujourd'hui.
12. Lisez le compteur après plusieurs incréments.
13. Que se passe-t-il si vous essayez de mettre total_passages = 100 directement ? (indice : un type counter ne se modifie que par incrément/décrément).

Niveau 5 — IN clause et bornes multiples

14. Listez les passages des stations S001 ET S002 le 15/09/2025 (indice : IN (...)).
15. Listez les passages d'une station sur 3 jours consécutifs (15, 16, 17 septembre).

Bonus

16. Essayez `SELECT * FROM station_passages WHERE event_type = 'rent';` — quel message d'erreur ?
Comment résoudre proprement ?
17. Quelle table créeriez-vous pour répondre à la question « tous les passages de type 'rent' d'une station un jour donné » sans `ALLOW FILTERING` ?

7.3. Conseils en cas de blocage

Si vous bloquez sur une requête :

1. **Vérifiez votre PRIMARY KEY** : la requête respecte-t-elle la contrainte (partition key complète + clustering key bien utilisée) ?
2. **Si Cassandra refuse**, c'est presque toujours que votre modélisation ne sert pas cette requête. Soit créez une nouvelle table adaptée, soit acceptez `ALLOW FILTERING` en sachant que c'est un mauvais signal.
3. **Documentation officielle CQL** : cassandra.apache.org/doc/latest/cassandra/cql/
4. **Cheat sheet en annexe** à la fin du document.

Partie 8 — Cours : anti-patterns et bonnes pratiques

8.1. Pourquoi un chapitre dédié ?

Cassandra punit lourdement les mauvaises pratiques. Une modélisation naïve qui marche en développement sur quelques milliers de lignes peut crasher un cluster en production sur des milliards de lignes. Connaître les anti-patterns dès le début vous évitera des nuits blanches.

8.2. Anti-pattern #1 : ALLOW FILTERING

Comme vu en partie 6, ALLOW FILTERING demande à Cassandra de scanner tout le cluster pour filtrer. Sur 10 000 lignes ça passe en dev. Sur 1 milliard ça crashe.

```
-- Mauvais : nécessite ALLOW FILTERING
SELECT * FROM station_passages WHERE event_type = 'rent' ALLOW FILTERING;

-- Bon : créer une table adaptée
CREATE TABLE station_passages_by_event_type (
  station_id text,
  day date,
  event_type text,
  passage_time timestamp,
  user_id text,
  PRIMARY KEY ((station_id, day, event_type), passage_time)
) WITH CLUSTERING ORDER BY (passage_time DESC);

SELECT * FROM station_passages_by_event_type
WHERE station_id = 'S001' AND day = '2025-09-15' AND event_type = 'rent';
```

Règle absolue : si vous voyez ALLOW FILTERING dans votre code, c'est un signal d'alarme. Soit votre modélisation est inadaptée (créez une nouvelle table), soit votre besoin n'est pas pour Cassandra (allez vers SQL ou Elasticsearch).

8.3. Anti-pattern #2 : la partition qui grossit indéfiniment

Une partition qui n'a pas de borne dans le temps grossit indéfiniment. Au bout d'un an, elle fait 50 Go et les requêtes timeout.

```
-- Mauvais : la partition d'une grosse station grossit sans limite
CREATE TABLE station_passages_bad (
  station_id text,
  passage_time timestamp,
  ...
  PRIMARY KEY (station_id, passage_time)
);
-- → Une station Part-Dieu avec 10 000 passages/jour fait 3,6M lignes/an dans 1 partition

-- Bon : bucketing par jour
CREATE TABLE station_passages_good (
  station_id text,
  day date,
  passage_time timestamp,
  ...
  PRIMARY KEY ((station_id, day), passage_time)
);
-- → Une partition = max 10 000 lignes (1 jour), reste manageable
```

8.4. Anti-pattern #3 : abus des secondary indexes

Cassandra propose des **secondary indexes (SI)**, mais ils sont **très limités** en performance. Chaque nœud doit consulter son index local, ce qui équivaut à un scan distribué.

```
-- À éviter (sauf cas très spécifiques)
CREATE INDEX ON users (city);

-- Cette requête semble naturelle mais est très inefficace à grande échelle
SELECT * FROM users WHERE city = 'Lyon';

-- Bon : créer une table dédiée à la requête
CREATE TABLE users_by_city (
  city text,
  user_id uuid,
  email text,
  PRIMARY KEY (city, user_id)
);
```

Quand un secondary index est-il OK ? Très basse cardinalité (peu de valeurs distinctes), requêtes rares, et vous savez exactement ce que vous faites. En cas de doute → table dédiée.

8.5. Anti-pattern #4 : raisonner comme en SQL

Les pièges classiques des développeurs SQL :

- **Vouloir faire des JOIN** — impossible en CQL, dénormaliser à la place
- **Vouloir des GROUP BY arbitraires** — très limité, pré-agréger en écriture
- **Vouloir des sous-requêtes** — n'existent pas
- **Croire que les transactions multi-lignes existent** — sauf LWT et BATCH limités
- **Vouloir une cohérence ACID forte par défaut** — configurable mais coûteux

8.6. Anti-pattern #5 : suppressions massives (tombstones)

Cassandra **ne supprime pas immédiatement les données** : elle écrit un « tombstone » (marqueur de suppression) qui sera physiquement nettoyé lors du compactage. **Conséquence** : supprimer des millions de lignes crée des millions de tombstones qui ralentissent les lectures suivantes.

Bonne pratique : pour les données éphémères, utilisez le TTL plutôt que DELETE.

INSERT INTO table (...) VALUES (...) USING TTL 86400; → expire dans 24h sans créer de tombstones lourds.

8.7. BATCH : utiliser correctement

BATCH permet d'écrire dans plusieurs tables de manière atomique. **À utiliser uniquement pour la cohérence multi-tables**, pas pour la performance (contrairement aux idées reçues).

```
-- Bon usage : écrire un passage dans 3 tables qui se reflètent
BEGIN BATCH
  INSERT INTO station_passages (...) VALUES (...);
  INSERT INTO user_passages (...) VALUES (...);
  UPDATE daily_station_stats SET total_passages = total_passages + 1
    WHERE station_id = '...' AND day = '...';
APPLY BATCH;

-- Mauvais usage : grouper des inserts indépendants pour la perf
```

```
BEGIN BATCH
INSERT INTO users (...) VALUES (alice...);
INSERT INTO users (...) VALUES (bob...);
INSERT INTO users (...) VALUES (carol...);
APPLY BATCH;
-- → Ralentit même les performances, à ne pas faire
```

Partie 9 — TP : agrégation, BATCH et compteurs

9.1. Limites de l'agrégation native CQL

CQL supporte quelques fonctions d'agrégation, mais **uniquement dans une partition** :

```
-- Marche : agrégation sur une partition
SELECT COUNT(*) FROM cityflow.station_passages
WHERE station_id = 'S001' AND day = '2025-09-15';

SELECT MIN(passage_time), MAX(passage_time)
FROM cityflow.station_passages
WHERE station_id = 'S001' AND day = '2025-09-15';

-- Marche aussi : sur plusieurs partitions via IN
SELECT COUNT(*) FROM cityflow.station_passages
WHERE station_id IN ('S001', 'S002') AND day = '2025-09-15';

-- Echoue : GROUP BY arbitraire
SELECT day, COUNT(*) FROM cityflow.station_passages GROUP BY day;
-- Group by is only supported on the columns of the PRIMARY KEY
```

Cassandra n'est pas une base analytique. Pour les vraies agrégations multi-partitions, on utilise **Apache Spark + le connecteur DataStax**, ou on **pré-agrège en écriture** dans des tables dédiées avec des compteurs.

9.2. TP — Pré-agrégation avec counters

VOTRE TOUR — Pré-agrégation et BATCH

Étape 1 : vérifier daily_station_stats

Vous avez déjà créé daily_station_stats dans la partie 6. Vérifiez sa structure :

```
DESCRIBE TABLE daily_station_stats;
```

Étape 2 : créer une seconde table d'agrégation

Créez hourly_station_stats qui compte les passages par station par heure :

```
CREATE TABLE hourly_station_stats (
  station_id text, day date, hour int,
  total_passages counter,
  PRIMARY KEY ((station_id, day), hour)
);
```

Étape 3 : écrire un passage dans 3 tables avec BATCH

Pour chaque nouveau passage, on doit incrémenter les 2 compteurs **et** insérer dans la table principale. Écrivez le BATCH qui fait les 3 opérations en une.

Étape 4 : tester en boucle

Exécutez votre BATCH 10 fois avec des données différentes. Puis lisez les compteurs.

Étape 5 : observation

Que se passerait-il si une des écritures du BATCH échouait ? Cassandra garantit-elle l'atomicité comme une transaction SQL ?

Réponse attendue dans `mes\_requetes\_cql.md` : non, le BATCH garantit une **isolation limitée** et n'est pas une vraie transaction ACID. C'est une cohérence à terme.

Partie 10 — Projet fil rouge CityFlow

10.1. Objectif

Vous avez maintenant toutes les compétences pour intégrer Cassandra dans votre architecture CityFlow. Cette partie est **collective** : faites-la avec votre équipe projet.

10.2. Rappel des user stories Cassandra (C1-C4)

ID	User Story
US-C1	En tant qu'analyste, je veux retrouver l'historique des passages d'une station spécifique sur une période donnée.
US-C2	En tant que système, je veux pouvoir enregistrer plusieurs milliers d'événements de passage par minute sans dégradation.
US-C3	En tant qu'utilisateur, je veux consulter mes propres connexions des 30 derniers jours (audit RGPD).
US-C4	En tant qu'analyste, je veux obtenir l'évolution journalière du nombre de passages pour identifier les pics d'affluence.

10.3. Étape 1 — Modélisation collective

VOTRE TOUR — Conception du keyspace cityflow

En équipe, sur papier ou tableau, modélisez l'ensemble des tables nécessaires pour les 4 user stories. Pour chaque table :

1. **Listez la requête qu'elle sert** (cette table répond à US-C...)
2. **Définissez la PRIMARY KEY** complète (partition key + clustering key)
3. **Justifiez** : pourquoi cette partition key ? Quelle est la taille max attendue d'une partition ?
4. **Précisez le tri** (CLUSTERING ORDER BY)

Points de réflexion à débattre en équipe :

- US-C1 et US-C4 nécessitent-elles la même table ? Ou deux tables différentes ?
- US-C2 (écritures massives) influence-t-elle votre choix ? Combien d'écritures par seconde votre modèle peut-il encaisser ?
- US-C3 (audit user) : embedding ou table dédiée par user ?
- Quelles tables d'agrégation (counters) prévoyez-vous pour US-C4 ?

Livable : un schéma complet sous forme de tableau ou DESCRIBE, présenté en 2 min au formateur.

10.4. Étape 2 — Création des tables CityFlow

VOTRE TOUR — Création du keyspace et des tables

Dans cassandra-web, ouvrez la console CQL et exécutez vos requêtes ; gardez en parallèle un fichier seed/cassandra/init.cql ouvert dans votre éditeur (VSCode) qui consigne tout le DDL.

1. Créez le keyspace cityflow (à côté de tp_cassandra qui sert pour l'apprentissage).
2. Créez **toutes les tables** issues de votre modélisation papier.
3. Validez chaque création par un DESCRIBE TABLE.
4. **Documentez chaque table** dans docs/modelisation-cassandra.md au fur et à mesure.

Le fichier init.cql devient votre script de seed réutilisable.

10.5. Étape 3 — Peupler avec un jeu de données réaliste

VOTRE TOUR — Insertion de données réalistes

Construisez un dataset CityFlow réaliste :

- **15 stations** différentes (S001 à S015) avec des noms de lieux Lyonnais
- **30 utilisateurs** (user_001 à user_030)
- **200 passages** répartis sur **7 jours consécutifs** (la semaine du 15/09/2025)
- Variez les event_type (rent/return), les heures, les couples user/station

Pensez aux **3 tables synchronisées via BATCH** : station_passages + user_passages + daily_station_stats. Sinon vos données sont incohérentes entre tables.

Astuce : plutôt que de coller 200 INSERTs un par un, écrivez une boucle ou un fichier .cql exécuté via cqlsh -f. Mais l'important c'est d'avoir les données, pas l'élégance du seed.

10.6. Étape 4 — Implémenter les 4 user stories

VOTRE TOUR — Les requêtes des user stories

Pour chaque user story, **écrivez la (ou les) requête CQL** qui la résolvent. Notez-les dans queries/cassandra-queries.md du repo CityFlow.

US-C1 : historique des passages d'une station sur une période

Une partition couvre 1 jour. Pour 7 jours, comment faire ? IN ou plusieurs requêtes ?

US-C2 : encaisser des milliers d'écritures/min

Pas une requête, mais une **démonstration** : insérez 100 passages d'affilée dans un BATCH ou en boucle, mesurez le temps. Cassandra encaisse-t-elle ?

US-C3 : audit utilisateur sur 30 jours

Sur quelle table ? Quelle est la requête ? Combien de partitions touchées ?

US-C4 : évolution journalière des passages d'une station

Utilisez votre table d'agrégation counter. Une seule requête doit suffire.

10.7. Étape 5 — Documenter et commiter

Pour finaliser cette séance et alimenter votre projet CityFlow :

- Dans votre repo Git, créez docs/modelisation-cassandra.md avec :
 - - Le DDL complet (CREATE KEYSPACE + CREATE TABLEs)
 - - La **justification de chaque PRIMARY KEY**
 - - L'estimation de taille de partition pour chaque table
- Créez seed/cassandra/init.cql (script CQL qui peuple les tables)
- Notez les 4 user stories résolues dans queries/cassandra-queries.md
- Faites un commit Git : feat(cassandra): modélisation et seed initial

À ce stade, vous avez :

- Une instance Cassandra fonctionnelle dans votre stack CityFlow
- Un keyspace cityflow avec 4-5 tables query-first
- Justifications écrites des choix de PRIMARY KEY
- Un jeu de données réaliste sur 7 jours
- 4 user stories implémentées
- Un commit Git versionné

C'est la moitié de votre note de projet CityFlow pour la partie Cassandra !

Annexe A — Cheat sheet cassandra-web

Action	Comment ?
Se connecter	http://localhost:8083 (sans auth — auto-connectée au conteneur Cassandra)
Lister les keyspaces	Page d'accueil affiche tous les keyspaces
Explorer une table	Cliquer sur le keyspace → cliquer sur la table
Voir les données d'une table	Vue par défaut quand on ouvre une table : aperçu paginé
Voir le schéma d'une table	Onglet Schema ou bouton info sur la table
Exécuter une requête CQL libre	Bouton Query / Execute + zone de saisie
Voir le résultat d'une requête	Affiché en tableau sous la zone de saisie
Rafraîchir la liste	Bouton Refresh ou recharger la page

Annexe B — Mémo des commandes CQL

B.1. Keyspaces et tables

```
CREATE KEYSPACE name
  WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};

USE keyspace;
DESCRIBE KEYSPACES;
DESCRIBE KEYSPACE name;
DESCRIBE TABLES;
DESCRIBE TABLE name;
DROP KEYSPACE name;

CREATE TABLE name (
  col1 type1, col2 type2, ...
  PRIMARY KEY ((part_key), cluster_key1, cluster_key2)
) WITH CLUSTERING ORDER BY (cluster_key1 ASC, cluster_key2 DESC);

ALTER TABLE name ADD column_name type;
ALTER TABLE name DROP column_name;
DROP TABLE name;
TRUNCATE name;
```

B.2. Types CQL

```
text varchar      -- chaîne UTF-8
int bigint smallint -- entiers
float double      -- nombres à virgule
boolean           -- true / false
uuid timeuuid    -- identifiants uniques
date time timestamp -- dates et heures
blob              -- binaire
list<T> set<T> map<K,V> -- collections
counter           -- compteur incrémental
frozen<T>         -- collection traitée comme valeur atomique
```

B.3. CRUD

```
-- INSERT (= UPSERT)
INSERT INTO table (col1, col2) VALUES (...) [USING TTL n];
```

```

-- UPDATE (= UPSERT aussi)
UPDATE table SET col = value WHERE pk = ?;

-- DELETE
DELETE FROM table WHERE pk = ?;
DELETE col FROM table WHERE pk = ?;

-- SELECT (strict)
SELECT cols FROM table
WHERE pk_col = ? [AND cluster_col operator ?]
[ORDER BY cluster_col ASC|DESC]
[LIMIT n];

-- Comptage et agrégations limitées (sur une partition)
SELECT COUNT(*) FROM table WHERE pk = ?;
SELECT MAX(col), MIN(col), AVG(col) FROM table WHERE pk = ?;

```

B.4. BATCH et compteurs

```

BEGIN BATCH
  INSERT INTO t1 (...) VALUES (...);
  UPDATE t2 SET ... WHERE ...;
APPLY BATCH;

-- Compteurs (table dédiée avec colonnes counter)
UPDATE table SET counter_col = counter_col + 1 WHERE pk = ?;
UPDATE table SET counter_col = counter_col - 1 WHERE pk = ?;

```

Annexe C — Pour aller plus loin

C.1. Utiliser cqlsh en ligne de commande

```

# Se connecter au conteneur Cassandra
docker exec -it cityflow-cassandra cqlsh

# Une fois dans cqlsh :
USE cityflow;
DESCRIBE TABLES;
SELECT * FROM station_passages LIMIT 10;

```

C.2. Connecter Cassandra à du code

```

# Python avec cassandra-driver
from cassandra.cluster import Cluster
cluster = Cluster(['localhost'])
session = cluster.connect('cityflow')
rows = session.execute("SELECT * FROM station_passages LIMIT 10")

// Node.js avec cassandra-driver
const cassandra = require('cassandra-driver');
const client = new cassandra.Client({
  contactPoints: ['localhost'],
  localDataCenter: 'dc1',
  keyspace: 'cityflow'
});

```

```
const result = await client.execute("SELECT * FROM station_passages LIMIT 10");

// .NET avec CassandraCSharpDriver
var cluster = Cluster.Builder().AddContactPoint("localhost").Build();
var session = cluster.Connect("cityflow");
var rows = session.Execute("SELECT * FROM station_passages LIMIT 10");
```

C.3. ScyllaDB : l'alternative haute performance

ScyllaDB est une réécriture de Cassandra en C++ (au lieu de Java), 10× plus rapide à ressources égales. **API 100% compatible** : si vous savez Cassandra, vous savez ScyllaDB. C'est ce que Discord utilise pour 5 trillions de messages.

C.4. Ressources recommandées

- **DataStax Academy** : datastax.com/learn — cours gratuits sur Cassandra et data modeling
- **Documentation officielle** : cassandra.apache.org/doc/latest/
- **Killrvideo** : tutoriel pratique full-stack avec Cassandra (killrvideo.com)
- **Cassandra: The Definitive Guide** (O'Reilly) — référence absolue

À retenir absolument de cette séance

- **Cassandra = BigTable + Dynamo** : modèle colonne + architecture peer-to-peer AP.
- **Pas de master** : tous les nœuds sont égaux, le cluster tolère bien les pannes.
- **PRIMARY KEY = (partition key) + clustering key** : la partition key détermine le nœud, la clustering key l'ordre.
- **Query-first design** : on modélise selon les requêtes, pas selon les données. Une table par type de requête.
- **Accepter la duplication** : la même donnée écrite dans plusieurs tables, c'est volontaire.
- **CQL ressemble à SQL mais ne fonctionne pas pareil** : pas de JOIN, pas de GROUP BY arbitraire, requêtes contraintes par la PRIMARY KEY.
- **ALLOW FILTERING = signal d'alarme** : votre modélisation est mauvaise, créez une nouvelle table.
- **Bucketing par temps** dans la partition key pour éviter les partitions trop grosses.
- **Pré-agrégé en écriture** avec des counters plutôt que d'agréger en lecture.